IF25-40401 · DEEP LEARNING · PERTEMUAN 06 · MATERI PENDAMPING STUDIO 2

HYPERPARAMETER TUNING UNTUK CNN & ViT

Learning rate, optimizer, scheduler, regularisasi, early stopping, strategi fine-tuning, dan pelacakan eksperimen dengan W&B

Tim Dosen Deep Learning · Institut Teknologi Sumatera (ITERA)

TUJUAN PEMBELAJARAN

Setelah mengikuti sesi ini, mahasiswa diharapkan mampu:

1. **Membedakan parameter dan hyperparameter** serta memprioritaskannya
2. **Merancang eksperimen yang jujur**: protokol evaluasi (hold-out, *k*-fold, multi-seed), satu kunci per eksperimen, hipotesis tertulis
3. **Memilih learning rate, optimizer, batch size, dan scheduler** berdasarkan kurva training
4. **Mendiagnosis underfitting/overfitting** dan memilih regularisasi serta early stopping yang tepat
5. **Membandingkan strategi fine-tuning** (scratch, linear probing, full FT) dan **inisialisasi bobot** pada CNN dan ViT
6. **Menjelaskan presisi numerik** (fp16 AMP, fp32, fp64) dan dampaknya pada kecepatan, memori, dan akurasi
7. **Melacak dan membandingkan eksperimen** dengan Weights & Biases

OUTLINE

1. Metodologi Eksperimen
2. Setup: Kode Eksperimen & Weights & Biases
3. Learning Rate, Optimizer & Batch Size
4. LR Scheduler & Warmup
5. Regularisasi & Early Stopping
6. Fine-tuning, Pilihan Model & Inisialisasi
7. Presisi Numerik
8. Strategi Pencarian & Resep Praktis

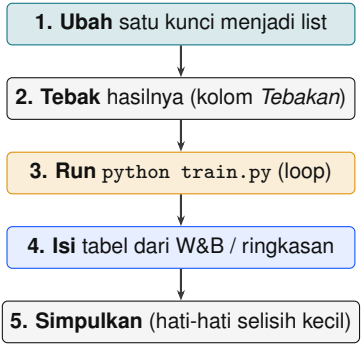
Model Sama, Satu Angka Berbeda

Tebak dulu

ResNet-18 pretrained, data sama, kode sama. Hanya `lr` yang diganti dari 10^{-3} ke 10^{-1}. Berapa poin akurasi yang hilang? **0? 5? 50?**

Kita **ukur sendiri**: tiap topik punya **kartu eksperimen** — pindah ke kode, run, kembali ke slide.

- nilai yang dicoba = **list** di `CONFIG`, mis. `optimizer=["adam", "adamw", "nadam"]`
- `python train.py` mencoba **semua** nilai di **beberapa arsitektur**
- ± 1 menit per run (RTX 5070, 10 epoch), semua tercatat di W&B



Kembalikan list ke satu nilai (baseline) sebelum pindah kartu.

Parameter vs Hyperparameter

Siapa yang menentukan nilainya?

	Parameter θ	**Hyperparameter**
Contoh	bobot kernel konvolusi, bias, γ/β BatchNorm, bobot attention	learning rate, optimizer, batch size, epoch, dropout, augmentasi, scheduler
Ditentukan oleh	**gradient descent** (otomatis)	**Anda** (sebelum training)
Jumlah	jutaan (ResNet-18: 11,2 M)	belasan (`CONFIG` di `train.py`)
Dipilih memakai	data **train**	data **val**

$$\underbrace{\boldsymbol{\theta}^*(\boldsymbol{\lambda}) = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{train}}(\boldsymbol{\theta}; \boldsymbol{\lambda})}_{\text{training (loop dalam)}} \qquad \underbrace{\boldsymbol{\lambda}^* = \arg\max_{\boldsymbol{\lambda}} \text{Skor}_{\text{val}}\left(\boldsymbol{\theta}^*(\boldsymbol{\lambda})\right)}_{\text{hyperparameter tuning (loop luar)}}$$

$\boldsymbol{\lambda}$ = vektor hyperparameter. Setiap evaluasi loop luar = satu training penuh $\Rightarrow$ mahal, jadi harus dirancang dengan cermat.

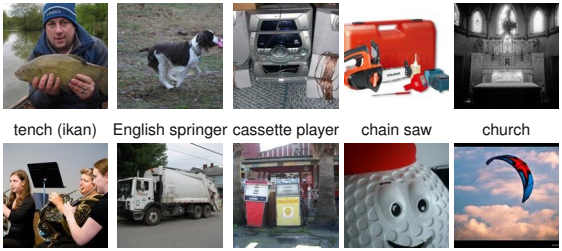
BAGIAN 01 · METODOLOGI

Eksperimen yang Jujur

Dataset, Protokol Evaluasi, Satu Kunci per Eksperimen

Dataset: Imagenette

10 kelas ImageNet yang mudah dibedakan



Satu contoh per kelas dari split `val`, versi 160 px.

- J. Howard (fast.ai, 2019): subset ImageNet untuk **eksperimen cepat**; versi **160 px** ($\pm$94 MB, unduh otomatis).
- **train** 9.469 (858–993/kelas) · **val** 3.925 (357–419/kelas): **hampir seimbang**, akurasi bermakna.
- `train` dari ImageNet train, `val` dari ImageNet val.

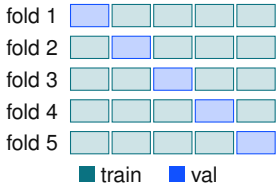
Konsekuensi

Bobot pretrained ImageNet **sudah melihat** kelas ini, jadi fine-tuning sangat mudah. Versi sulit: **Imagewoof** (10 ras anjing).

Protokol Evaluasi

Dari yang termurah sampai yang paling bisa dipercaya

5-fold cross-validation



Setiap data jadi val tepat sekali; skor = **rata-rata** $\pm$ **std** dari 5 run. **Stratified**: proporsi kelas di tiap fold sama dengan data aslinya.

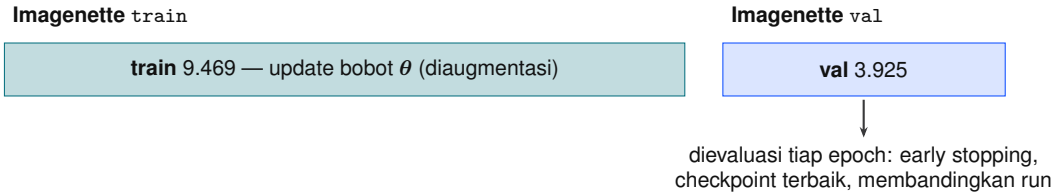
Protokol	Kapan dipakai	Biaya
Train / val tetap	iterasi cepat, demo kelas **(kode kita)**	$1\times$
Train / val / test	butuh angka akhir yang jujur (paper, laporan)	$1\times$
k-fold CV (k=5)	data sedikit; butuh error bar	$5\times$
Stratified k-fold	+ kelas tidak seimbang (mis. data medis, sampah)	$5\times$
Multi-seed	mengukur noise init, urutan batch, augmentasi	$n\times$

Bisa digabung

Stratified 5-fold $\times$ 3 seed = 15 run per konfigurasi. Laporkan **mean** $\pm$ **std**; selisih yang lebih kecil dari std belum tentu nyata.

Pilihan Kita: Train/Val Tetap

Demi waktu kelas: satu split, satu seed



Kenapa cukup ini?

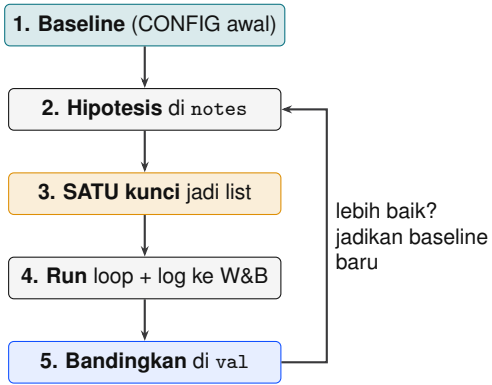
Satu run ± 1 menit; stratified 5-fold $\times$ 3 seed membuat setiap kartu **15$\times$ lebih lama**. Split tetap + `seed=42` membuat semua run bisa dibandingkan dan muat di satu sesi kelas.

Harganya (bias)

- skor val sedikit optimistis: val juga dipakai memilih
- satu seed: selisih kecil (mis. $<$0,5 poin) bisa kebetulan

Studio 2: **validation set tersembunyi** milik asisten = test set.

Siklus Eksperimen



Aturan main

- **Satu kunci divariasikan per eksperimen** (boleh ditambah `model`) — dua kunci sekaligus = efek tak bisa diatribusikan, dan jumlah run berlipat.
- **Seed tetap** (`seed=42`), split tetap.
- **Catat semuanya**, termasuk run yang gagal.
- **Satu seed** demi waktu: selisih kecil belum tentu temuan; konfirmasi dengan multi-seed / *k*-fold di luar kelas.
- Hipotesis ditulis **sebelum** run, bukan setelah melihat hasil.

Mana yang Dituning Lebih Dulu?

Prioritas untuk fine-tuning pretrained

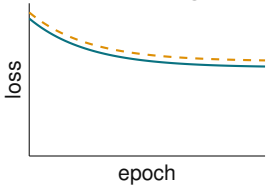
Prio.	Hyperparameter	Kenapa	Kunci di `CONFIG`
1	Learning rate	paling sensitif; meleset satu orde = gagal	`lr`, `backbone_lr_mult`
1	Pretrained & strategi FT	menentukan titik awal optimasi	`finetune`
2	Optimizer & weight decay	berinteraksi kuat dengan lr	`optimizer`, `weight_decay`
2	Scheduler & epoch	bentuk lr sepanjang training	`scheduler`, `warmup_epochs`
2	Batch size	noise gradien & lr optimal	`batch_size`
3	Regularisasi	penting saat data sedikit / overfit	`augment`, `dropout`
3	Early stopping	hemat waktu, checkpoint terbaik	`patience`, `monitor`
4	Arsitektur & resolusi	ganti setelah resep training stabil	`model`, `img_size`
–	Presisi numerik	kecepatan & memori, bukan akurasi	`precision`

Urutan mengikuti Google *Deep Learning Tuning Playbook* dan Smith (2018): mulai dari efek terbesar, kunci nilainya, lalu lanjut.

Diagnosis Dulu, Baru Tuning

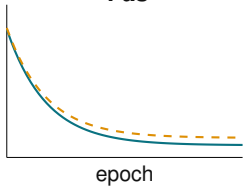
loss train (garis) vs val (putus-putus)

Underfitting



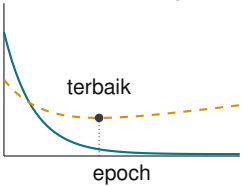
- keduanya tinggi & datar
- ↑ lr / epoch, pretrained, kurangi regularisasi, model lebih besar

Pas



- turun bersama, gap kecil
- pertahankan; coba resolusi / model lain

Overfitting



- train terus turun, val naik
- augmentasi, dropout, wd, label smoothing, MixUp, **early stopping**

BAGIAN 02 · SETUP

Kode Eksperimen & W&B

Tiga File, CONFIG Berisi List, Semua Run Tercatat

Lima Backbone

timm, pretrained ImageNet

- varian **teringan** tiap keluarga: 3 CNN klasik, 1 CNN modern, 1 Transformer
- bobot ImageNet-1k (ViT-Tiny: ImageNet-21k $\rightarrow$ 1k)
- head 1000 kelas diganti head **10 kelas acak**

Kenapa Imagenette, bukan data sampah? Agar konfigurasi terbaik untuk kompetisi tetap menjadi temuan kelompok Anda sendiri.

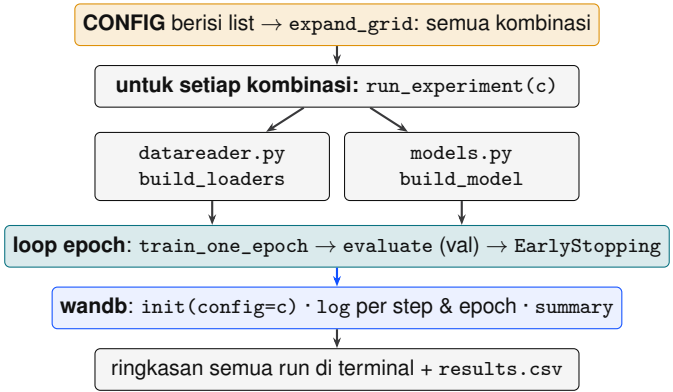
`model`	**Keluarga**	**Param**	**s/epoch***	**10 ep***
`resnet18`	CNN residual	11,2 M	4	$\pm$0,8 menit
`mobilenetv3`	depthwise + SE	1,5 M	5	$\pm$0,9 menit
`efficientnet_b0`	compound scaling	4,0 M	7	$\pm$1,3 menit
`convnext_atto`	CNN modern	3,4 M	5	$\pm$1,0 menit
`vit_tiny`	Transformer	5,5 M	5	$\pm$0,9 menit

*terukur di RTX 5070 (batch 64, 160 px, AMP, 8 worker), termasuk validasi 3.925 citra; epoch pertama $\pm$5 s lebih lambat. Bottleneck-nya **loading data di CPU**, bukan ukuran model: MobileNetV3 (1,5 M) tidak lebih cepat dari ResNet-18 (11,2 M). Semua: varian **teringan** di keluarganya, satu file per arsitektur di `architectures/`, head 10 kelas via `timm.create_model(..., num_classes=10)`.

Anatomi Kode


```
hyperparameter-tuning/
+- train.py
|      CONFIG, loop, training, W&B
+- datareader.py
|      Imagenette → DataLoader
+- models.py
|      pilih arsitektur + strategi FT
+- architectures/
     +- resnet18.py
     +- mobilenetv3.py
     +- efficientnet_b0.py
     +- convnext_atto.py
     +- vit_tiny.py
```

Tiap arsitektur: satu file berisi `build(...)`. Tanpa argparse: edit `CONFIG`, simpan, run. Run divergen ditandai `diverged`, loop lanjut.



Blok CONFIG

List = dicoba satu per satu

```
CONFIG = dict(
    # --- Data ---
    img_size=[160],          # 128 | 160 | 224
    train_fraction=[1.0],    # 0.1 = 10% data train
    augment=["basic"],       # none | basic | strong
    mixup_alpha=[0.0], cutmix_alpha=[0.0],
    # --- Model ---
    model=["resnet18", "vit_tiny"],
    finetune=["full_ft"],    # scratch | linear_probe
    init=["default"],        # khusus scratch: kaiming
                             # | xavier | normal
    backbone_lr_mult=[0.1], # lr backbone = lr * 0.1
    dropout=[0.0], drop_path=[0.0],
    # --- Optimizer ---
    optimizer=["adamw"],     # sgd | adam | adamw
                             # | nadam | rmsprop
    lr=[1e-3], weight_decay=[0.05],
    batch_size=[64], epochs=[10],
    label_smoothing=[0.0],
    # --- LR scheduler ---
    scheduler=["cosine"],    # none | step |
     cosine
                             # | onecycle |
     plateau
    warmup_epochs=[1],
    # --- Early stopping ---
    patience=[3],            # None = tanpa ES
    # --- Presisi numerik ---
    precision=["fp16_amp"], # fp32 | fp64
    # --- Tetap (bukan list) ---
    seed=42,
    monitor="val_loss",      # val_acc | val_f1
    wandb_project="if25-p6-hyperparameter",
    wandb_mode="online",     # offline | disabled
    notes="",                # tulis HIPOTESIS!
)
```

Jumlah run = **perkalian panjang semua list** (di atas: 2 run). Nama run & group W&B otomatis, mis. `vit_tiny-optimizer=nadam`.

W&B: Buku Catatan Eksperimen


```
# sekali per run, di dalam loop kombinasi
run = wandb.init(
    project="if25-p6-hyperparameter",
    name="vit_tiny-optimizer=nadam",
    group="optimizer",
    config=c,   # SEMUA hyperparameter run ini
    notes="nadam ~ adam?")
# per 10 step
wandb.log({"train/loss": loss, "train/lr": lr,
           "train/grad_norm": gnorm}, step=step)
# per epoch
wandb.log({"epoch": ep, "val/loss": vl,
           "val/acc": va, "val/f1": vf})
# akhir run: metrik di epoch terbaik
run.summary.update({"best/val_acc": acc})
wandb.log({"val/confusion_matrix":
           wandb.plot.confusion_matrix(...)})
wandb.finish()
```

Di dashboard, gunakan:

- **Runs table**: urutkan menurut `best/val_acc`, tampilkan kolom config.
- **Group** / **filter**: group `optimizer`, tag `vit_tiny`.
- **Parallel coordinates**: hyperparameter mana yang berkorelasi dengan skor.
- **Line plot** `train/lr`: cek bentuk scheduler.
- **Report**: rangkuman untuk laporan Studio 2.

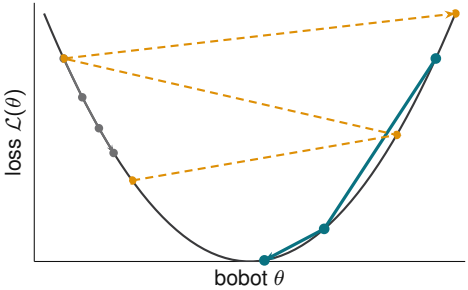
Tanpa internet: `wandb_mode="offline"`, lalu `wandb sync`.

BAGIAN 03 · OPTIMASI

Learning Rate & Optimizer

Hyperparameter Paling Sensitif

Learning Rate: Ukuran Langkah



$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

- η **kecil**: tiap langkah aman tetapi pendek $\Rightarrow$ belum konvergen saat epoch habis (**underfit**).
- η **pas**: turun cepat lalu stabil.
- η **besar**: melompati lembah, loss berosilasi atau **meledak** (NaN). `train.py` mendeteksinya dan menghentikan run (`diverged=True` di W&B).
- lr optimal bergantung pada optimizer, batch size, dan backbone $\Rightarrow$ ganti salah satunya = cek lr ulang.

Ilustrasi 1 dimensi; loss sebenarnya berdimensi jutaan.

Kartu 1: Sweep Learning Rate

Skala log, bukan linear

List di CONFIG

```
lr=[1e-5, 1e-4, 1e-3, 1e-2, 1e-1]
model=["resnet18", "vit_tiny"]
# 5 x 2 = 10 run, ±10 menit
```

Hipotesis (tulis di `notes`)

- Akurasi vs $\log_{10} \eta$ berbentuk **U terbalik**, untuk kedua arsitektur.
- lr terlalu besar: berhenti cepat (divergen / early stopping).
- lr terbaik ViT **belum tentu** sama dengan ResNet.

		ResNet-18		ViT-Tiny	
`lr`	**Tebakan**	**Henti ep**	**Val acc**	**Henti ep**	**Val acc**
10^{-5}					
10^{-4}					
10^{-3}					
10^{-2}					
10^{-1}					

Setelah orde terbaik ketemu, haluskan dengan faktor $\pm 3\times$: `lr=[3e-4, 1e-3, 3e-3]`.

Amati di W&B: group `lr`; `train/loss` per step dan `val/acc`; panel *Parallel coordinates* `lr` → `best/val_acc`.

SGD, Momentum, dan Adam

Aturan update yang dipilih `optimizer`

SGD + momentum (`"sgd"`, `momentum=0.9`)

$$\mathbf{v}_t = \mu\,\mathbf{v}_{t-1} + \mathbf{g}_t, \qquad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta\,\mathbf{v}_t$$

- satu lr untuk semua bobot; momentum meredam zig-zag
- butuh lr lebih besar (umumnya 10^{-2}) dan lebih sensitif terhadap lr
- sering sedikit lebih baik dalam generalisasi bila dituning cermat

Adam (`"adam"`): lr adaptif per bobot

$$\mathbf{m}_t = \beta_1\mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \quad \mathbf{s}_t = \beta_2\mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}$$

- bobot dengan gradien kecil mendapat langkah relatif besar
- lr default 10^{-3} biasanya “langsung jalan”
- `"nadam"` (Dozat, 2016): Adam + momentum **Nesterov** — $\hat{\mathbf{m}}_t$ “melihat ke depan” satu langkah
- `"rmsprop"`: hanya $\mathbf{s}_t$, tanpa $\mathbf{m}_t$ dan koreksi bias

Kartu 2: Optimizer

Setiap optimizer punya lr idealnya

List di CONFIG

```
optimizer=["sgd", "adam", "adamw",
           "nadam", "rmsprop"]
model=["resnet18", "vit_tiny"]
# 5 x 2 = 10 run, ±10 menit
```

Hipotesis (tulis di `notes`)

- Pada lr 10^{-3}, SGD tertinggal: butuh lr $\sim 10\times$ lebih besar.
- NAdam $\approx$ Adam; AdamW unggul tipis (wd 0,05 decoupled).

Lanjutan: `optimizer=["sgd"]`, `lr=[1e-3, 1e-2]`.

Optimizer	wd	Tebakan	Val acc ResNet-18	Val acc ViT-Tiny
SGD+mom.	0,05 L2			
Adam	0,05 L2			
AdamW	0,05 dec.			
NAdam	0,05 L2			
RMSprop	0,05 L2			
SGD, lr 10^{-2}	0,05 L2			

Semua lr 10^{-3} kecuali baris terakhir. L2 = decay lewat gradien; dec. = decoupled (AdamW). `momentum=0.9` untuk SGD & RMSprop.

Amati di W&B: group `optimizer`; kurva `val/acc` per optimizer di satu panel, pisahkan per tag `model`.

Adam + L2 $\neq$ AdamW

Loshchilov & Hutter (2019)

Adam + L2 (`"adam"`, `weight_decay=`λ): decay masuk ke gradien

$$\mathbf{g}_t \leftarrow \nabla\mathcal{L} + \lambda\boldsymbol{\theta} \;\Rightarrow\; \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}$$

$\lambda\boldsymbol{\theta}$ ikut **dinormalisasi** oleh $\sqrt{\hat{\mathbf{s}}_t}$: bobot dengan gradien kecil di-decay sangat kuat, yang gradiennya besar hampir tidak.

AdamW (`"adamw"`): decay **dipisah** dari gradien

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\Big(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon} + \lambda\boldsymbol{\theta}\Big)$$

Semua bobot menyusut dengan laju yang sama $\eta\lambda$ — perilaku yang memang diinginkan dari weight decay.

Kartu 2b: dua list sekaligus

```
optimizer=["adam", "adamw"]
weight_decay=[0.0, 0.05]
# 2 x 2 x 2 model = 8 run
```

- $\lambda = 0$: Adam dan AdamW **identik**
- $\lambda = 0{,}05$: Adam + L2 vs AdamW

Jika Adam + L2 paling rendah: λ yang wajar untuk AdamW terlalu kuat sebagai L2.

Di PyTorch: `torch.optim.Adam(weight_decay=...)` = L2; `torch.optim.AdamW` = decoupled.

Kartu 3: Batch Size

Goyal et al. (2017): linear scaling rule

$$B \to kB \quad \Rightarrow \quad \eta \to k\,\eta$$

Batch $4\times$ lebih besar = langkah $4\times$ lebih sedikit per epoch; kompensasi dengan lr $4\times$ lebih besar (perlu warmup).

- Batch besar: gradien lebih akurat, GPU efisien, tetapi **lebih sedikit update**.
- Batch kecil: gradien berisik (efek regularisasi), lebih lambat.
- `CUDA out of memory`: turunkan `batch_size` *dan* lr.

Hipotesis: $256 + 4 \cdot 10^{-3}$ menyamai baseline ($64 + 10^{-3}$); $256 + 10^{-3}$ tertinggal.

List di CONFIG

```
batch_size=[32, 64, 256]
lr=[1e-3, 4e-3]
model=["resnet18"] # 3 x 2 = 6 run, ±6 menit
```

Batch	Step/ep	Tebakan	s/epoch	Val acc, lr	
				10^{-3}	$4 \cdot 10^{-3}$
32	295				
64 (baseline)	147				
256	36				

Step/ep = $\lfloor 9.469/B \rfloor$ (batch terakhir yang tidak penuh dibuang). Dua list = grid 3×2: kombinasi lain ikut dicoba sebagai bonus. Ada waktu? tambahkan `"vit_tiny"`.

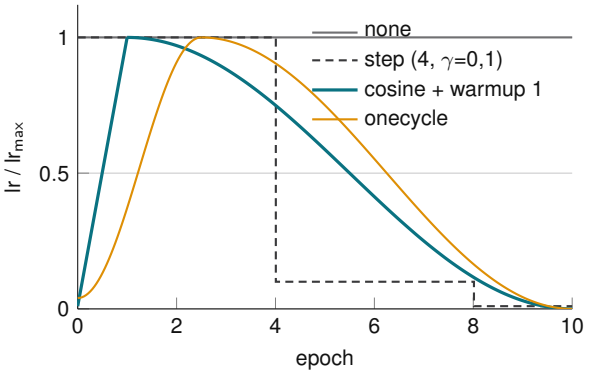
BAGIAN 04 · SCHEDULER

LR Scheduler & Warmup

Learning Rate Tidak Harus Konstan

Bentuk-Bentuk Scheduler

lr relatif terhadap `lr` di CONFIG, 10 epoch



- **step**: turun $\times\gamma$ tiap `step_size` (= 4) epoch; sederhana, tapi waktu turunnya harus ditebak.
- **cosine**: turun mulus ke `min_lr`; default modern (ConvNeXt, ViT).
- **onecycle**: naik lalu turun; lr puncak boleh lebih besar.
- **plateau**: turun $\times\gamma$ **hanya jika** `val_loss` macet `plateau_patience` epoch — reaktif, tidak terjadwal.

Di `train.py` semua di-`step()` per batch, kecuali plateau (per epoch).

Kenapa Warmup?

Melindungi bobot pretrained di awal training

- Di awal, **head baru diinisialisasi acak** $\Rightarrow$ loss besar, gradien besar dan "berisik".
- Gradien itu mengalir ke backbone; dengan lr penuh, fitur pretrained yang bagus bisa rusak di beberapa ratus langkah pertama.
- Adam: estimasi $\hat{\mathbf{s}}_t$ belum stabil pada langkah awal.
- **Warmup linear**: lr naik dari 1% ke 100% selama `warmup_epochs`.

$$\eta_t = \eta_{\max}\left(0{,}01 + 0{,}99\,\frac{t}{T_{\text{warm}}}\right), \quad t < T_{\text{warm}}$$

Kombinasi yang dipakai di `train.py`

`LinearLR` (warmup) $\rightarrow$ `CosineAnnealingLR` via `SequentialLR`, di-step setiap batch.

- ViT umumnya **lebih membutuhkan** warmup daripada CNN dengan BatchNorm.
- `backbone_lr_mult=0.1` adalah perlindungan kedua: backbone bergerak $10\times$ lebih pelan dari head.

Kartu 4: Scheduler

Efeknya bergantung pada lr

List di CONFIG

```
scheduler=["none", "step", "cosine",
           "onecycle", "plateau"]
model=["resnet18", "vit_tiny"]
# 5 x 2 = 10 run, ±10 menit
```

4b, lr kebesaran: `lr=[1e-2]`, `scheduler=["none", "cosine"]`, `warmup_epochs=[0, 1]` (8 run)

Hipotesis (tulis di `notes`)

- Pada lr yang sudah pas, scheduler mengubah sedikit.
- Pada lr kebesaran, cosine + warmup **menyelamatkan** run, terutama ViT.

Scheduler	lr	Tebakan	Val acc	
			ResNet-18	ViT-Tiny
none	10^{-3}			
step (4, 0,1)	10^{-3}			
cosine, warmup 1	10^{-3}			
onecycle	10^{-3}			
plateau	10^{-3}			
none, warmup 0	10^{-2}			
none, warmup 1	10^{-2}			
cosine, warmup 0	10^{-2}			
cosine, warmup 1	10^{-2}			

Plateau hanya turun jika `val_loss` macet 2 epoch: dalam 10 epoch bisa saja **tidak pernah** turun.

Amati di W&B: `train/lr` per step (bentuk kurva) dan `val/loss`.

BAGIAN 05 · REGULARISASI

Regularisasi & Early Stopping

Menutup Gap antara Train dan Val

Kartu 5: Memancing Overfitting

10% data (≈800 citra), tanpa regularisasi

List di CONFIG

```
train_fraction=[0.1]
augment=["none"]
weight_decay=[0.0]
patience=[None] # tanpa ES
epochs=[30]
model=["resnet18", "vit_tiny"]
```

Hipotesis (tulis di `notes`)

- Akurasi train mendekati 100%, val tertinggal jauh: **gap** besar.
- Loss train terus turun; loss val mendatar atau naik.

	ResNet-18		ViT-Tiny	
Epoch	**Train acc**	**Val acc**	**Train acc**	**Val acc**
5				
10				
20				
30				

Data train ±950 citra: run cepat dan menjadi **baseline Kartu 6**. Studio 2 (±3.700 citra) juga termasuk "data sedikit".

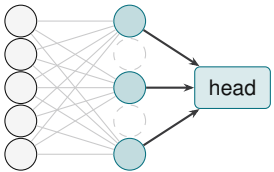
Amati di W&B: `epoch/train_acc` vs `val/acc` dan `epoch/gap_acc`.

Dropout & Stochastic Depth

`dropout` dan `drop_path`

Dropout (Srivastava et al., 2014): matikan neuron acak

$$\tilde{\mathbf{h}} = \frac{\mathbf{m} \odot \mathbf{h}}{1-p}, \qquad m_i \sim \text{Bernoulli}(1-p)$$



Di timm: diterapkan **sebelum classifier** (`drop_rate`). Saat `eval()` dropout otomatis mati.

Stochastic depth / **drop path** (Huang et al., 2016): matikan **seluruh cabang residual**

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \frac{b_l}{1-p} F_l(\mathbf{x}_l), \quad b_l \sim \text{Bernoulli}(1-p)$$

- jaringan “acak lebih dangkal” tiap batch $\Rightarrow$ ensemble implisit
- standar di ConvNeXt & ViT (`drop_path` 0,1–0,2)
- didukung kelima model di `architectures/` (timm `drop_path_rate`)

Weight Decay & Label Smoothing

`weight_decay` dan `label_smoothing`

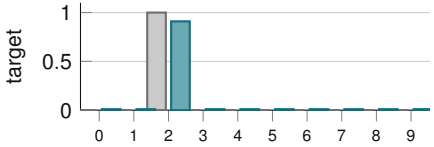
Weight decay: menarik bobot ke nol

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \frac{\lambda}{2}\|\boldsymbol{\theta}\|^2 \;\Rightarrow\; \boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\,\boldsymbol{\theta} - \eta\,\mathbf{g}$$

- setara untuk SGD; tidak setara untuk Adam (lihat bagian optimizer)
- AdamW: λ wajar 0,01–0,1; SGD: 10^{-4}–$5 \cdot 10^{-4}$

Label smoothing (Szegedy et al., 2016): target tidak lagi 100% yakin

$$y_k^{\text{LS}} = (1 - \varepsilon)\, y_k + \frac{\varepsilon}{K}$$



abu-abu: one-hot · teal: $\varepsilon = 0{,}1$, $K = 10$. Mencegah logit membesar tanpa batas; model lebih terkalibrasi.

Augmentasi Data, MixUp & CutMix

`augment`, `mixup_alpha`, `cutmix_alpha`

`augment`	Transformasi train
`none`	resize + center crop (sama dengan val)
`basic`	`RandomResizedCrop`(0,35–1) + flip horizontal
`strong`	`basic` + `TrivialAugmentWide` + `RandomErasing`(p=0,25)

Citra val **tidak pernah** diaugmentasi (`datareader.py`). Augmentasi harus masuk akal untuk domain: flip vertikal untuk foto sampah? Mungkin. Untuk angka "6" vs "9"? Tidak.

MixUp (Zhang et al., 2018): campur dua citra *dan* labelnya

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j, \quad \tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j$$

dengan $\lambda \sim \text{Beta}(\alpha, \alpha)$, α = `mixup_alpha`.
CutMix (Yun et al., 2019): tempel potongan citra *j* ke citra *i*; λ = proporsi luas yang tersisa.

- label menjadi **soft** $\Rightarrow$ `CrossEntropyLoss` menerima probabilitas
- diterapkan per batch lewat `collate_fn` (`v2.MixUp`, `v2.CutMix`)
- butuh **lebih banyak epoch** agar manfaatnya terlihat

Kartu 6: Regularisasi

List di CONFIG (satu per satu)

```
augment=["none", "basic", "strong"]
# kembalikan augment=["none"], lalu:
dropout=[0.0, 0.5]
weight_decay=[0.0, 0.05]
label_smoothing=[0.0, 0.1]
mixup_alpha=[0.2], cutmix_alpha=[1.0]
```

Hipotesis (tulis di `notes`)

- Augmentasi paling ampuh saat data sedikit.
- Gap mengecil $\neq$ val acc pasti naik.

Perubahan	Tebakan	ResNet-18 Gap*	ResNet-18 Val	ViT-Tiny Gap*	ViT-Tiny Val
(Kartu 5)					
aug. basic					
aug. strong					
dropout 0,5					
wd 0,05					
label smooth. 0,1					
MixUp + CutMix					

*train − val di epoch terakhir; dengan augmentasi/MixUp akurasi train dihitung pada citra teraugmentasi, jadi gap bisa negatif. List berisi nilai Kartu 5 (mis. `0.0`) membuat baris baseline ikut dijalankan ulang. Dengan data sedikit, satu seed makin rawan kebetulan.

Amati di W&B: `epoch/gap_acc` semua run dalam satu panel.

Kartu 7: Early Stopping

`patience`, `min_delta`, `monitor`

Aturan (kelas `EarlyStopping` di `train.py`): berhenti jika

$$\underbrace{m_t > m^* - \delta}_{\text{tidak membaik}} \quad \text{selama } P \text{ epoch berturut-turut}$$

- m = `monitor` (`val_loss` turun; `val_acc`/`val_f1` naik), δ = `min_delta`, P = `patience`
- setiap kali membaik: **simpan checkpoint**; hasil run = metrik di epoch terbaik, *bukan* epoch terakhir
- Studio 2 dinilai dengan macro-F1 $\Rightarrow$ `monitor="val_f1"`

List di CONFIG

```
patience=[None, 5, 2] # None = penuh
epochs=[30]
model=["resnet18"] # 3 run, ±6 menit
```

`patience`	**None**	**5**	**2**
Epoch dijalankan			
Epoch terbaik			
Waktu (menit)			
Val acc terbaik			

Hipotesis: val acc terbaik hampir sama, waktu jauh lebih singkat; P terlalu kecil berhenti di plateau sementara.

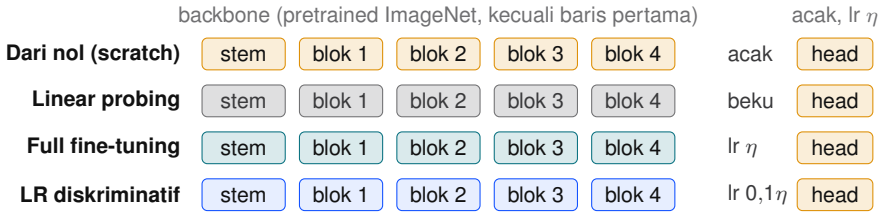
BAGIAN 06 · FINE-TUNING

Fine-tuning, CNN vs ViT & Init

Titik Awal, Bagian yang Dilatih, dan Seberapa Cepat

Empat Strategi Melatih Backbone

Bagian mana yang belajar?



Strategi	CONFIG	Cocok ketika	Risiko
Scratch	`finetune="scratch"`	domain jauh + data banyak	lambat, haus data
Linear probing	`finetune="linear_probe"`	data sangat sedikit	fitur tak beradaptasi
Full FT	`"full_ft"`, `backbone_lr_mult=1.0`	data cukup	fitur rusak bila lr besar
LR diskriminatif	`"full_ft"`, `backbone_lr_mult=0.1`	default yang aman	+1 hyperparameter

Kartu 8: Strategi Fine-tuning

ResNet-18 & ViT-Tiny, data penuh

List di CONFIG

```
finetune=["scratch", "linear_probe",
          "full_ft"]
model=["resnet18", "vit_tiny"]
# 3 x 2 = 6 run, ±6 menit
```

Lanjutan: `finetune=["full_ft"]`, `backbone_lr_mult=[1.0, 0.1]` (4 run).

Hipotesis (tulis di `notes`)

- Scratch tertinggal **jauh**: bobot pretrained adalah hyperparameter terpenting.
- Linear probe ≈ full FT di Imagenette, karena kelasnya *adalah* kelas ImageNet.

Strategi	Dilatih (R18)	Tebakan	Val acc	
			ResNet-18	ViT-Tiny
Scratch	11,2 M			
Linear probe	5 K			
Full FT $\times 1$	11,2 M			
Full FT $\times 0{,}1$	11,2 M			

$\times 1$ / $\times 0{,}1$ = `backbone_lr_mult`. Data sampah berbeda domain dari ImageNet ⇒ ulangi perbandingan ini di data Anda.

Amati di W&B: `val/acc` epoch 1: pretrained langsung tinggi, scratch mulai dari ≈10%.

Kartu 9: CNN vs ViT

Resep sama, backbone berbeda

List di CONFIG

```
model=["resnet18", "mobilenetv3",
       "efficientnet_b0",
       "convnext_atto", "vit_tiny"]
train_fraction=[1.0, 0.1]
# 5 x 2 = 10 run, ±8 menit
```

Hipotesis (tulis di `notes`)

- "ViT haus data" berlaku untuk training **dari nol**. Dengan pretraining ImageNet-21k, apakah ViT tetap kalah di 10% data?
- Ganti backbone = **cek lr ulang**: lr 10^{-3} belum tentu cocok untuk ViT.

Model	Param	Tebakan	Val 100%	Val 10%
ResNet-18	11,2 M			
MobileNetV3-S	1,5 M			
EfficientNet-B0	4,0 M			
ConvNeXt-Atto	3,4 M			
ViT-Tiny/16	5,5 M			

CNN: lokalitas & translation equivariance "dibangun" di konvolusi. ViT: attention global antar-patch 16×16; di 160 px = 10×10 patch (`img_size=[160]`). Run 10% data $<$1 menit.

Amati di W&B: `best_epoch`; jika terbaik di epoch 1–2, lr kemungkinan kebesaran.

Inisialisasi Bobot

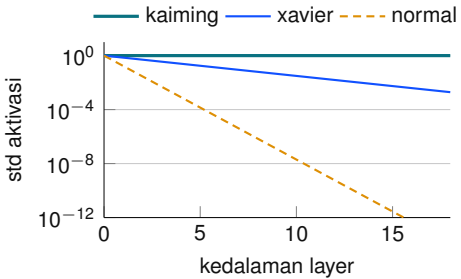
Hanya berarti saat melatih dari nol

Satu layer $y = \sum_{i=1}^{n_{\text{in}}} w_i x_i$:

$$\mathrm{Var}(y) = n_{\text{in}} \,\mathrm{Var}(w)\,\mathrm{Var}(x)$$

`init`	$\mathrm{Var}(w)$	Ide
`normal`	$0{,}01^2$	tetap, abaikan ukuran layer
`xavier`	$\frac{2}{n_{\text{in}}+n_{\text{out}}}$	jaga varians maju & mundur
`kaiming`	$\frac{2}{n_{\text{in}}}$	ReLU membuang separuh $\Rightarrow \times 2$

`default` = bawaan timm (umumnya Kaiming untuk CNN, truncated normal untuk ViT). Bobot pretrained **tidak** ditimpa: `init` hanya berlaku untuk `finetune="scratch"`.



Ilustrasi teoretis: jaringan polos ReLU tanpa normalisasi, $n_{\text{in}} = n_{\text{out}} = 3 \cdot 3 \cdot 64 = 576$. Sinyal yang mengecil ke nol = **vanishing**, gradien ikut hilang.

BatchNorm/LayerNorm & residual membuat jaringan modern **jauh lebih toleran** terhadap init.

Kartu 10: Inisialisasi Bobot

Dari nol, tanpa bobot pretrained

List di CONFIG

```
finetune=["scratch"]
init=["default", "kaiming",
      "xavier", "normal"]
model=["resnet18", "vit_tiny"]
# 4 x 2 = 8 run, ±7 menit
```

Hipotesis (tulis di `notes`)

- `normal` (std tetap 0,01) paling buruk: sinyal mengecil di tiap layer.
- ResNet-18 (BatchNorm + residual) **toleran**; ViT lebih sensitif.
- Semua jauh di bawah pretrained (Kartu 8).

		Val acc	
`init`	**Tebakan**	**ResNet-18**	**ViT-Tiny**
`default` (timm)			
`kaiming`			
`xavier`			
`normal` (0; 0,01)			
pretrained (Kartu 8)			

Satu seed: selisih <1 poin belum tentu nyata. `init` selain `default` dengan `finetune` pretrained $\Rightarrow$ `ValueError` (bobot pretrained tidak ditimpa).

Amati di W&B: `train/loss` beberapa ratus step pertama; init buruk = loss lama tertahan di $\ln 10 \approx 2{,}3$.

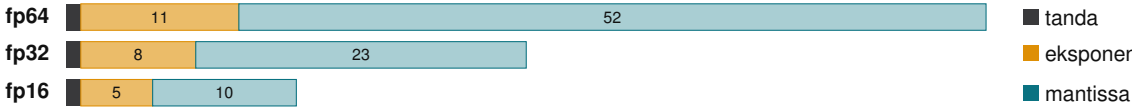
BAGIAN 07 · PRESISI

fp16 AMP vs fp32 vs fp64

Berapa Bit yang Benar-Benar Dibutuhkan?

Format Bilangan Floating-Point

tanda · eksponen (rentang) · mantissa (ketelitian)



	maks	ketelitian ε	byte/bobot
fp64	$1{,}8 \cdot 10^{308}$	$2{,}2 \cdot 10^{-16}$	8
fp32	$3{,}4 \cdot 10^{38}$	$1{,}2 \cdot 10^{-7}$	4
fp16	65.504	$9{,}8 \cdot 10^{-4}$	2

ε = selisih terkecil terhadap 1. Di fp16, $1 + 0{,}0001 = 1$: update kecil hilang. Bilangan $< 6 \cdot 10^{-8}$ menjadi 0 (**underflow**).

Di GPU

- fp32: standar training sejak awal deep learning
- fp16: dijalankan **Tensor Core**, memori aktivasi separuh
- fp64: simulasi ilmiah; GPU consumer (RTX 5070) hanya 1/64 throughput fp32

bf16 (8/7 bit): rentang fp32, populer untuk LLM; tidak kita uji.

Mixed Precision (AMP)

Micikevicius et al. (2018)

1. **autocast**: konvolusi & matmul dihitung di fp16; operasi sensitif (softmax, loss, reduksi) tetap fp32.
2. **Bobot master fp32**: update $\eta\, g \approx 10^{-6}$ akan hilang jika bobot disimpan di fp16.
3. **GradScaler**: loss dikali S (mis. 2^{16}) sebelum backward agar gradien kecil tidak underflow; dibagi S lagi sebelum `step`. Jika muncul Inf/NaN: step dilewati, S diperkecil.

`fp16_amp` hanya aktif di GPU CUDA; di MPS/CPU sama dengan fp32. fp64 tidak didukung MPS.

```
# train.py, train_one_epoch (fp16_amp)
scaler = torch.amp.GradScaler("cuda")
with torch.autocast("cuda", dtype=torch.float16):
    logits = model(x)
    loss = loss_fn(logits, y)
opt.zero_grad()
scaler.scale(loss).backward()  # loss * S
scaler.unscale_(opt)           # gradien / S
scaler.step(opt)               # lewati jika Inf
scaler.update()                # atur S

# fp64: model.to(device, torch.float64)
#       x.to(device, torch.float64)
```

Kartu 11: Presisi Numerik

Seberapa berpengaruh? Hasil benchmark RTX 5070

List di CONFIG

```
precision=["fp16_amp", "fp32",
           "fp64"]
model=["resnet18", "vit_tiny"]
epochs=[3] # 6 run
```

fp64 $\pm$2 menit/epoch; demo cepat: `python benchmark_precision.py` ($<$1 menit).

- **Akurasi**: praktis sama ($\leq$0,15 poin).
- **fp16 AMP**: 1,7$\times$ lebih cepat di GPU, memori $-$30%.
- **fp64**: 20–30$\times$ lebih lambat, memori 2$\times$, tanpa manfaat.

Model	Presisi	citra/s[a]	s/epoch[b]	Memori[b]	Val acc[b]
ResNet-18	fp16 AMP	**3.778**	**4,2**	**0,65 GB**	0,9541
	fp32	2.248	6,1	0,95 GB	0,9544
	fp64	80	140,7	1,79 GB	0,9557
ViT-Tiny	fp16 AMP	**4.286**	**4,9**	**0,70 GB**	0,9801
	fp32	2.454	5,5	1,10 GB	0,9789
	fp64	104	106,6	2,36 GB	0,9799

[a] `benchmark_precision.py`: train step, batch 64, 160 px, data sintetis (murni GPU). [b] `train.py`: 3 epoch data penuh + validasi; median s/epoch, memori GPU puncak, val acc terbaik.

Di `train.py` keunggulan fp16 mengecil (ResNet 1,45$\times$, ViT 1,1$\times$) karena **loading data di CPU** jadi bottleneck. fp32 sudah memakai TF32 untuk konvolusi (bawaan PyTorch).

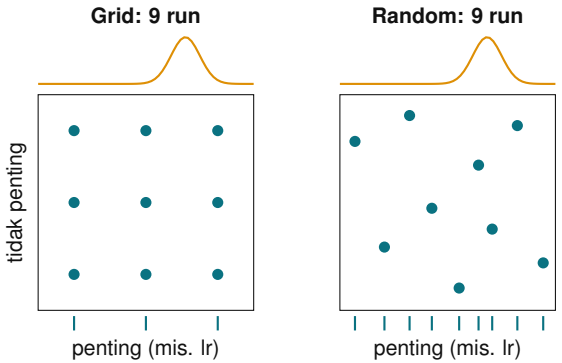
BAGIAN 08 · PENCARIAN

Strategi Pencarian & Resep

Dari Tebakan Manual ke Pencarian Sistematis

Grid Search vs Random Search

Bergstra & Bengio (2012)



- Grid: 9 run, tetapi hanya 3 nilai berbeda untuk hyperparameter yang penting.
- Random: 9 run $\Rightarrow$ 9 nilai berbeda untuk setiap dimensi.
- Kurva oranye = skor vs hyperparameter penting; puncaknya sempit dan mudah terlewat oleh grid.
- Sampling lr dalam **skala log**: $\eta = 10^{u}$, $u \sim \mathcal{U}(-5, -1)$.
- List di `CONFIG` kita = **grid search**: cocok untuk 1–2 kunci, mahal untuk banyak kunci.

Coarse-to-Fine & W&B Sweeps

Langkah berikutnya setelah tuning manual

Coarse-to-fine

1. **Kasar**: lr $\in \{10^{-5}, \ldots, 10^{-1}\}$, epoch pendek, `train_fraction=0.3`.
2. **Sempit**: sekitar 2–3 nilai terbaik, data penuh.
3. **Halus**: hyperparameter prioritas 2–3 pada lr terbaik.
4. **Konfirmasi**: ulangi konfigurasi terbaik dengan 2–3 seed.

List di `CONFIG` di kelas = langkah 1–2. W&B Sweeps mengotomatiskan langkah 2–3.

W&B Sweeps (Bayesian / random / grid)

```
sweep = {
  "method": "random",        # "bayes" | "grid"
  "metric": {"name": "best/val_acc",
             "goal": "maximize"},
  "parameters": {
    "lr": {"distribution": "log_uniform_values",
           "min": 1e-5, "max": 1e-2},
    "dropout": {"values": [0.0, 0.2, 0.5]},
    "optimizer": {"values": ["sgd", "adamw"]},
  },
}
sweep_id = wandb.sweep(sweep, project="if25-p6")
wandb.agent(sweep_id, function=..., count=20)
```

Agar bisa di-sweep, `run_experiment` perlu membaca nilai dari `wandb.config` setelah `wandb.init()`, bukan dari `CONFIG`.

Resep Praktis untuk Studio 2

1. **Baseline** starter kit + split lokal tetap (stratified); bila waktu cukup, ukur noise dengan 2–3 seed.
2. **Pretrained**; bandingkan **linear probe** vs **full FT** di data sendiri (Kartu 8).
3. **Sweep lr** dalam skala log; ganti optimizer/batch/backbone $\Rightarrow$ cek lr ulang.
4. **AdamW** (bukan Adam + L2), cosine + warmup 1 epoch, `backbone_lr_mult` 0,1.
5. Lihat kurva: gap besar $\Rightarrow$ augmentasi dulu, lalu label smoothing / MixUp / dropout.
6. **Early stopping** pada `val_f1` (metrik kompetisi), patience 3–5 (lebih besar untuk OneCycle).
7. Baru setelah itu: **backbone lain** & resolusi lebih besar.

Jangan

- mengubah 3 hal sekaligus
- memilih model memakai skor preview kompetisi
- mengklaim selisih $<$ noise
- lupa mencatat run yang gagal

Bawa pulang

Bandingkan tabel kartu Anda dengan tabel prioritas di awal: **mana yang benar-benar berefek** melebihi noise?

Aktivitas Kelas

30 menit, per kelompok Studio 2

1. Buat proyek W&B kelompok; isi `wandb_entity`.
2. Jalankan **baseline** `train.py` apa adanya (catat `best/val_acc`).
3. Setiap anggota memilih **satu** hyperparameter dari tabel prioritas, menulis hipotesis di `notes`, lalu mengisinya sebagai list 2–3 nilai.
4. Bandingkan di dashboard: siapa yang mengalahkan baseline melebihi noise?
5. Terapkan temuan terbaik ke **data kompetisi sampah** (`train_baseline.py` di starter kit).

Dikumpulkan (Studio 2)

- tautan **W&B Report**: tabel run, kurva val, hipotesis vs hasil
- 3 kalimat: hyperparameter apa yang paling berpengaruh pada data sampah, dan buktinya

Tips laptop (CPU/MPS): `model=["mobilenetv3"]`, `train_fraction=[0.3]`, `epochs=[5]`. GPU (RTX 5070): 10 epoch $\pm$1 menit per run.

Referensi (1/3)

- L. N. Smith. A Disciplined Approach to Neural Network Hyper-Parameters: Part 1 — Learning Rate, Batch Size, Momentum, and Weight Decay. *arXiv:1803.09820*, 2018.
- D. P. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.
- T. Dozat. Incorporating Nesterov Momentum into Adam. *ICLR Workshop*, 2016.
- I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019.
- I. Loshchilov, F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. *ICLR*, 2017.
- P. Goyal et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. *arXiv:1706.02677*, 2017.
- L. N. Smith, N. Topin. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. *arXiv:1708.07120*, 2017.
- J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- V. Godbole, G. E. Dahl, J. Gilmer, C. J. Shallue, Z. Nado. Deep Learning Tuning Playbook. `https://github.com/google-research/tuning_playbook`, 2023.

Referensi (2/3)

- N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR* 15, 2014.
- G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Weinberger. Deep Networks with Stochastic Depth. *ECCV*, 2016.
- C. Szegedy et al. Rethinking the Inception Architecture for Computer Vision. *CVPR*, 2016.
- H. Zhang, M. Cisse, Y. Dauphin, D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization. *ICLR*, 2018.
- S. Yun et al. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. *ICCV*, 2019.
- S. G. Müller, F. Hutter. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation. *ICCV*, 2021.
- S. Kornblith, J. Shlens, Q. V. Le. Do Better ImageNet Models Transfer Better? *CVPR*, 2019.
- A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.
- A. Steiner et al. How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers. *TMLR*, 2022.
- R. Wightman. PyTorch Image Models (timm) · J. Howard. Imagenette · Weights & Biases Docs. `https://docs.wandb.ai`

Referensi (3/3)

- R. Kohavi. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *IJCAI*, 1995.
- X. Bouthillier et al. Accounting for Variance in Machine Learning Benchmarks. *MLSys*, 2021.
- X. Glorot, Y. Bengio. Understanding the Difficulty of Training Deep Feedforward Neural Networks. *AISTATS*, 2010.
- K. He, X. Zhang, S. Ren, J. Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *ICCV*, 2015.
- P. Micikevicius et al. Mixed Precision Training. *ICLR*, 2018.
- J. Howard. Imagenette: A Smaller Subset of 10 Easily Classified Classes from ImageNet. `https://github.com/fastai/imagenette`, 2019.

Terima Kasih

Kode: `materials/hyperparameter-tuning/` (`train.py` · `datareader.py` · `models.py`) · Kompetisi: Studio Proyek 2 — Klasifikasi Sampah

Tim Dosen Deep Learning · Institut Teknologi Sumatera (ITERA)